

Computational Statistics

Importance Sampling

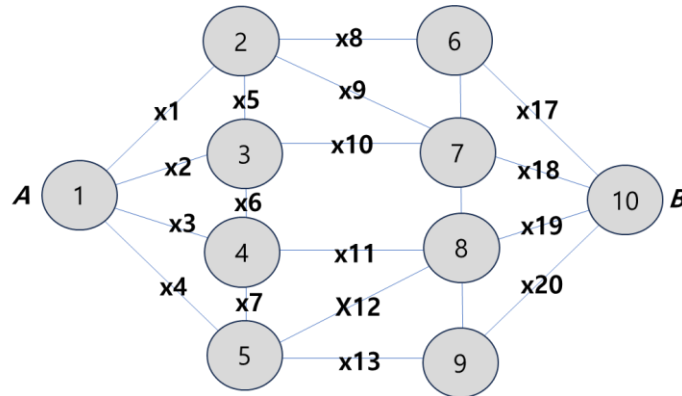
Network failure problem

232STG01

권가운

1. Importance Sampling – Network failure algorithm - Write the function $h(x)$

1.A problem description:



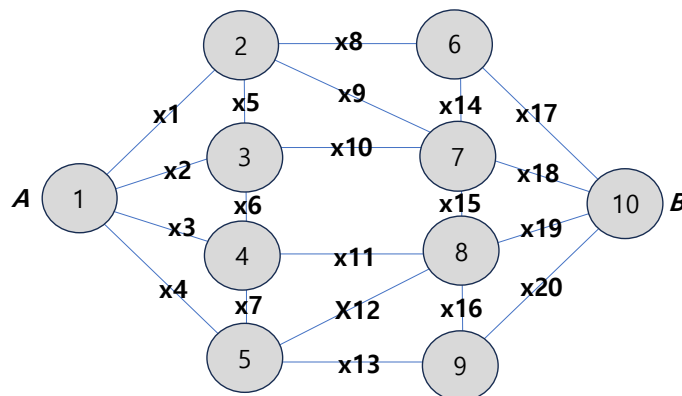
위 그림과 같이 A에서 B로 연결된 네트워크에서 네트워크 간 두 노드의 연결여부를 확인한다. 이를 위해 일부 edge가 손상된 상황에서도 연결 여부를 알 수 있는 unique한 함수 $h(x)$ 를 작성한다.

각 edge가 끊어질 확률이 $p=0.05$ 일 때 끊어질 확률(μ)을 추정하는 과정에서 naive MC(MC) 방법과 Importance Sampling(IS) 방법을 비교하고 실제로 IS 방법이 분산을 줄여주는지 확인한다.

먼저 naive MC방법을 이용해 μ_{MC} 를 추정하는 과정을 100번간 시뮬레이션을 한다. 이후 Importance Sampling(IS)를 이용하여 p 를 $p^*=0.2$ 로 바꾸어 μ_{IS} 계산하는 과정을 100번간 시뮬레이션 한다. 그리고 각 μ_{MC} 100개와 μ_{IS} 100개의 평균과 분산, 평균적으로 몇 번의 fail이 일어나는지를 비교한다.

더 나아가 compstat홈페이지의 "엄수연's $h(x)$ "와 작성한 $h(x)$ 의 100번의 simulation을 하는데 걸린 시간을 비교한다.

1.B Idea:



위 그림과 같이 A에서 B로 연결된 네트워크에서 네트워크 간 두 노드의 연결 여부를 확인하는 간단한 방법으로 statcomp홈페이지의 행렬에 너비 우선 탐색 방법(Breadth-First Search, BFS)을 결합하여 unique $h(x)$ 함수를 구현해보았다.

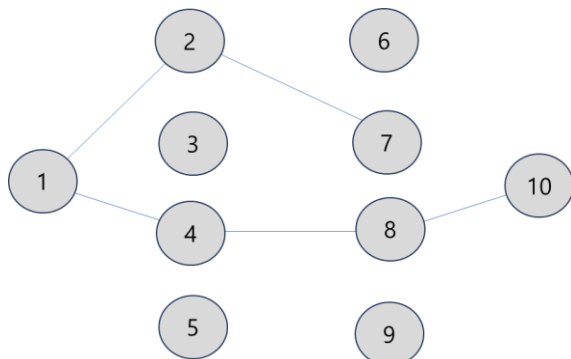
너비 우선 탐색 방법(Breadth-First Search, BFS)은 시작 노드에 인접한 노드부터 탐색하는 알고리즘으로 인접한 노드를 반복적으로 큐(Queue, 대기열)에 삽입하고 먼저 삽입된 노드부터 차례로 큐에서 꺼내는 방식을 사용한다.

너비 우선 탐색 방법의 동작 과정은 다음과 같다.

1. 탐색 시작 노드를 큐(Queue)에 삽입하고 방문 처리한다.
2. 큐(Queue)에서 노드를 꺼낸 뒤 방문하지 않은 인접 노드 정보를 모두 큐(Queue)에 삽입하고 방문 처리한다.
3. 2번의 과정을 더 이상 수행할 수 없을 때까지(큐(Queue)에 남은 노드가 없을 때까지) 반복한다.

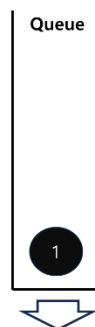
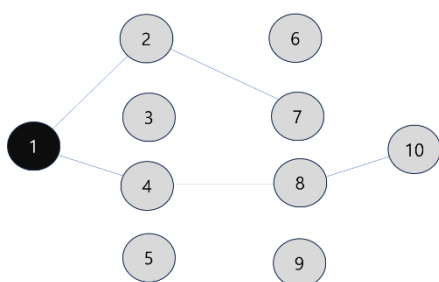
여기서 1번의 '방문 처리'란 탐색한 노드를 재방문하지 않는 것을 의미한다. 즉, 큐(Queue)에 한번이라도 삽입된 노드는 다시 삽입하지 않는다.

간단한 예제를 사용하여 구현되는 원리를 설명하겠다.

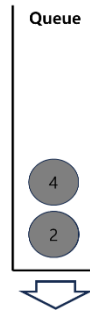
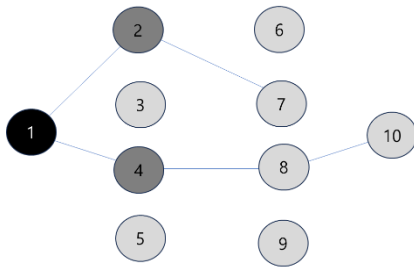


예제는 왼쪽 그림과 같이 연결 되어있다. Start node는 1로 설정하고 target node는 10으로 설정한다. 1번 노드와 10번 노드가 연결되어 있으므로 "연결 되어있다(FALSE)"를 return해야 한다.

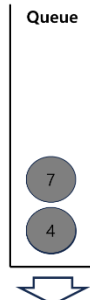
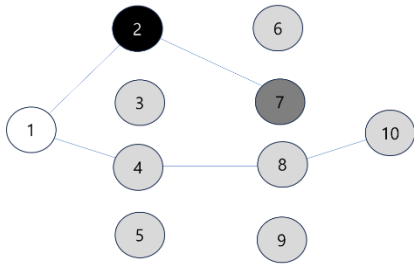
<동작 원리> (편의상 current 노드를 검정색으로, current 노드의 인접한 노드(neighborhood) 혹은 큐에 남아있는 노드를 진한 회색으로, 방문 처리된 노드는 흰색으로 표시하였다.)



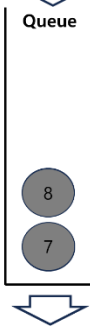
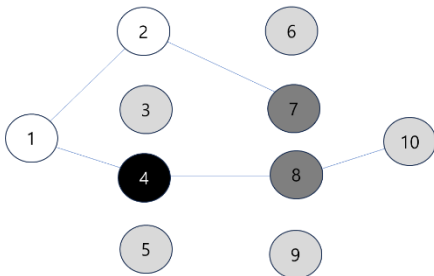
(1) 시작 노드인 노드 1을 큐에 삽입하고 방문 처리한다.



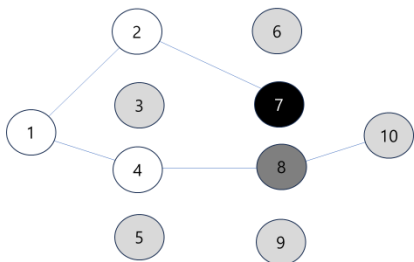
(2) 큐에서 노드 1을 꺼내고 인접한 노드 2와 4를 큐에 삽입하고 방문 처리한다.



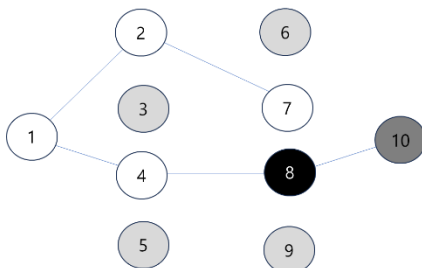
(3) 큐에서 노드 2를 꺼내고 인접한 노드 7을 큐에 삽입하고 방문 처리한다. (노드 1은 이미 방문처리 되었으므로 큐에 추가되지 않는다.)



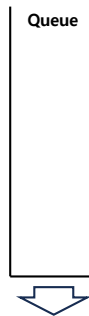
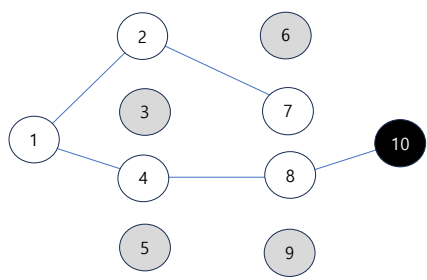
(4) 큐에서 노드 4를 꺼내고 인접한 노드 8을 큐에 삽입하고 방문 처리한다. (노드 1은 이미 방문처리 되었으므로 큐에 추가되지 않는다.)



(5) 큐에서 노드 7을 꺼내고 방문 처리한다. (노드 2는 이미 방문처리 되었으므로 큐에 추가되지 않는다.)



(6) 큐에서 노드 8을 꺼내고 인접한 노드 10을 큐에 삽입하고 방문 처리한다. (노드 4는 이미 방문처리 되었으므로 큐에 추가되지 않는다.)

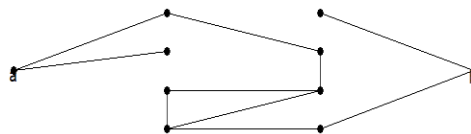


(7) 큐에 남아있던 노드 10을 꺼낸다. Current 노드와 target 노드가 노드 10으로 같으므로 루프는 멈추고 "연결되어 있음(FALSE)"을 return한다.

루프는 큐에 노드가 존재할 때 까지만 실행되므로 큐에 남은 노드가 없는데 target 노드에 도달하지 못하였다면 "연결되지 않음(TRUE)"를 return한다.

1.C results:

Example 1:

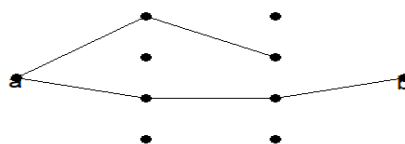


Input:

X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	X11	X12	X13	X14	X15	X16	X17	X18	X19	X20
1	1	0	0	0	0	1	0	1	0	1	1	1	0	1	0	1	0	0	1

Return: FALSE (연결되어 있음)

Example 2:

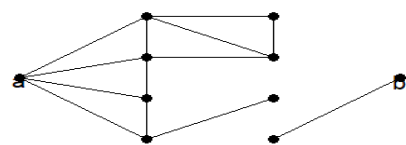


Input:

X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	X11	X12	X13	X14	X15	X16	X17	X18	X19	X20
1	0	1	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	1	0

Return: FALSE (연결되어 있음)

Example 3:

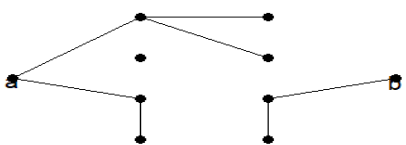


Input:

X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	X11	X12	X13	X14	X15	X16	X17	X18	X19	X20
1	1	1	1	1	1	1	1	1	1	0	1	0	1	0	0	0	0	0	1

Return: TRUE (연결되어 있지 않음)

Example 4:



Input:

X1	X2	X3	X4	X5	X6	X7	X8	X9	X10	X11	X12	X13	X14	X15	X16	X17	X18	X19	X20
1	0	1	0	0	0	1	1	1	0	0	0	0	0	0	1	0	0	1	0

Return: TRUE (연결되어 있지 않음)

2. Importance Sampling – Network failure algorithm - Simulation

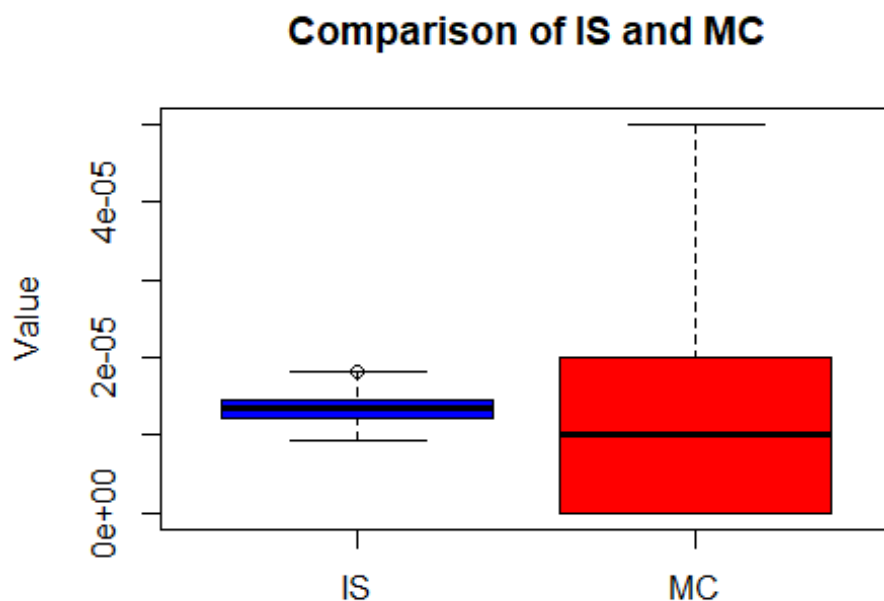
2.A problem description:

너비 우선 탐색 방법을 사용해 구현한 $h(x)$ 는 아이디어가 이해하기 쉽고, 확장이 용이하며, start 노드와 target 노드의 변경이 자유롭다는 장점이 있다. 반면, 모든 연결된 노드를 탐색하고 loop를 사용하기 때문에 노드의 수가 많아지면 실행속도가 느려질 것으로 예상되었다.

따라서 홈페이지에 있는 엄수연's $h(x)$ 와 구현한 $h(x)$ 로 naive MC방법을 이용해 μ_{MC} 를 추정하는 과정을 100번간 시뮬레이션을 한 뒤, Importance Sampling(IS)를 이용하여 p 를 $p^*=0.2$ 로 바꾸어 μ_{IS} 계산하는 과정을 100번간 시뮬레이션 하였다. 두 $h(x)$ 를 사용하였을 때 시뮬레이션에 걸린 수행시간을 비교해보았다.

또한, 각 μ_{MC} 100개와 μ_{IS} 100개의 평균과 분산 및 평균적으로 몇 번의 fail이 일어나는지를 확인하였다.

2.B results:



sample size는 10^5 개로 하여 μ_{MC} 추정치를 100번 구하고 μ_{IS} 추정치를 100번 구한 뒤 각각 μ_{MC} , μ_{IS} 값들의 boxplot을 그려본 결과는 위와 같았다. 그림에서도 쉽게 Monte – Carlo 방법을 이용한 경우보다 Importance Sampling 방법을 이용하였을 때 분산이 작은 것을 확인할 수 있었다.

μ_{MC} 추정하는 과정을 100번 시행하여 구한 100개 값의 평균, 분산 및 failure 횟수와 p 를 $p^*(0.2)$ 로 바꾸어 μ_{IS} 계산하는 과정을 100번간 시행한 뒤 구한 100개 값의 평균, 분산 및 failure

횃수를 비교한 결과는 다음과 같다.

	Mean	Var	Number of Failure
MC	1.3e-05	1.727273e-10	1.3
IS	1.351035e-05	3.845595e-12	484.32

추정 결과 μ_{MC} , μ_{IS} 의 평균 값은 매우 유사하지만 분산은 Importance Sampling 방법을 사용하여 구한 값이 Monte-Carlo 방법을 이용해 구한 값보다 작았다. 반면, 평균 failure 횃수는 1.3회에서 484.32회로 증가하였다. 이를 통해 Importance Sampling을 사용하면 사건이 일어날 확률을 조정하여 rare event의 분산을 줄일 수 있다는 것을 확인할 수 있었다.

	MC	IS
엄수연's h(x)	646.01 sec	824.08 sec
My h(x)	1594.30 sec	1445.75 sec

한편, 홈페이지 엄수연's h(x)와 구현한 h(x)의 100번의 μ_{MC} , μ_{IS} 를 구하는 simulation의 수행시간은 위와 같았다. 두 방법 모두 my h(x)가 시뮬레이션에 약 두 배의 시간이 걸린 것을 확인할 수 있었다. 특히 Monte-Carlo 방법의 경우 두 배 이상의 시간이 걸린 것을 확인할 수 있었는데, 이는 $p=0.05$ 로 매우 적은 확률로 각 edge가 끊어져 loop를 더 오래 실행했기 때문으로 예상된다.

3. APPENDIX – CODE

```
myplot<-function(x){
plot(c(0,1,1,1,1,2,2,2,2,3),c(0,1.5,0.5,-0.5,-1.5,1.5,0.5,-0.5,-1.5,0),axes=FALSE,xlab="",ylab="",pch=16)
text(0,-0.1,"a")
text(3,-0.1,"b")
if(x[1]==1){lines(c(0,1),c(0,1.5))}
if(x[2]==1){lines(c(0,1),c(0,0.5))}
if(x[3]==1){lines(c(0,1),c(0,-0.5))}
if(x[4]==1){lines(c(0,1),c(0,-1.5))}
if(x[5]==1){lines(c(1,1),c(1.5,0.5))}
if(x[6]==1){lines(c(1,1),c(0.5,-0.5))}
if(x[7]==1){lines(c(1,1),c(-0.5,-1.5))}
if(x[8]==1){lines(c(1,2),c(1.5,1.5))}
if(x[9]==1){lines(c(1,2),c(1.5,0.5))}
if(x[10]==1){lines(c(1,2),c(0.5,0.5))}
if(x[11]==1){lines(c(1,2),c(-0.5,-0.5))}
if(x[12]==1){lines(c(1,2),c(-1.5,-0.5))}
if(x[13]==1){lines(c(1,2),c(-1.5,-1.5))}
if(x[14]==1){lines(c(2,2),c(1.5,0.5))}
if(x[15]==1){lines(c(2,2),c(0.5,-0.5))}
```

```

if(x[16]==1){lines(c(2,2),c(-0.5,-1.5))}
if(x[17]==1){lines(c(2,3),c(1.5,0))}
if(x[18]==1){lines(c(2,3),c(0.5,0))}
if(x[19]==1){lines(c(2,3),c(-0.5,0))}
if(x[20]==1){lines(c(2,3),c(-1.5,0))}
}

```

```
myplot(x=rep(1,20));
```

unique code for h(x)

```

is_connected <- function(x, start_node=1, target_node=10) {

  graph <- matrix(c(
    1,x[1],x[2],x[3],x[4],0,0,0,0,0,
    x[1],1,x[5],0,0,x[8],x[9],0,0,0,
    x[2],x[5],1,x[6],0,0,x[10],0,0,0,
    x[3],0,x[6],1,x[7],0,0,x[11],0,0,
    x[4],0,0,x[7],1,0,0,x[12],x[13],0,
    0,x[8],0,0,0,1,x[14],0,0,x[17],
    0,x[9],x[10],0,0,x[14],1,x[15],0,x[18],
    0,0,0,x[11],x[12],0,x[15],1,x[16],x[19],
    0,0,0,0,x[13],0,0,x[16],1,x[20],
    0,0,0,0,0,x[17],x[18],x[19],x[20],1),
    ncol=10,byrow=T)

  visited <- rep(FALSE, ncol(graph))
  queue <- integer(0)

  visited[start_node] <- TRUE
  queue <- append(queue, start_node)

  while (length(queue) > 0) {
    current_node <- queue[1]
    queue <- queue[-1]

    if (current_node == target_node) {
      return(FALSE) # 노드 1 과 10 이 연결됨
    }

    neighbors <- which(graph[current_node, ] == 1) # 현재 노드와 연결된 노드
    for (neighbor in neighbors) {
      if (!visited[neighbor]) { # 방문하지 않았던 노드만 방문후 queue 에 추가
        visited[neighbor] <- TRUE
        queue <- append(queue, neighbor)
      }
    }
  }

  return(TRUE) # 노드 1 과 10 이 연결되지 않음
}

```

```

x1 <- c(1,1,0,0,0,0,1,0,1,0,1,1,1,0,1,0,1,0,0,1)
is_connected(x1);

## [1] FALSE

myplot(x1);

x2 <- c(1,0,1,0,0,0,0,0,1,0,1,0,0,0,0,0,0,0,1,0)
is_connected(x2);

## [1] FALSE

myplot(x2);

x3 <- c(1,1,1,1,1,1,1,1,1,1,0,1,0,1,0,0,0,0,0,1)
is_connected(x3);

## [1] TRUE

myplot(x3);

x4 <- c(1,0,1,0,0,0,1,1,1,0,0,0,0,0,0,1,0,0,1,0)
is_connected(x4);

## [1] TRUE

myplot(x4)

```

비교용 $h(x)$ (엄수연's $h(x)$)

```

myroute<-function(x){
m<-matrix(c(      1,x[1],x[2],x[3],x[4],0,0,0,0,0,
      x[1],1,x[5],0,0,x[8],x[9],0,0,0,
      x[2],x[5],1,x[6],0,0,x[10],0,0,0,
      x[3],0,x[6],1,x[7],0,0,x[11],0,0,
      x[4],0,0,x[7],1,0,0,x[12],x[13],0,
      0,x[8],0,0,0,1,x[14],0,0,x[17],
      0,x[9],x[10],0,0,x[14],1,x[15],0,x[18],
      0,0,0,x[11],x[12],0,x[15],1,x[16],x[19],
      0,0,0,0,x[13],0,0,x[16],1,x[20],
      0,0,0,0,0,x[17],x[18],x[19],x[20],1),ncol=10,byrow=T)
res<-ifelse((m%%m%%m%%m%%m%%m%%m%%m%%m%%m%%m)[1,10]>0,0,1)
return(res)
}

```

simulation

```

## MC with 엄수연's  $h(x)$ 

nsim <- 10^5
p <- 0.05
X <- matrix(nrow = nsim, ncol = 20)

mu_MC_vector <- numeric(100)
num_fail <- numeric(100)

```

```

total_time <- system.time({
  for (i in 1:100) {
    set.seed(i)

    for (j in 1:nsim) {
      X[j, ] <- rbinom(20, 1, 1 - p)
    }
    # number of failure
    num_fail[i] <- sum(apply(X, 1, myroute))

    # mu_MC
    mu_MC_vector[i] <- num_fail[i] / nsim
  }
})

#mu_MC_vector
total_time;

## 사용자 시스템 elapsed
## 258.38    2.89  646.01

mean(num_fail);

## [1] 1.3

mean(mu_MC_vector);

## [1] 1.3e-05

var(mu_MC_vector)

## [1] 1.727273e-10

## MC with my h(x) (is_connected(x))

nsim <- 10^5
p <- 0.05
X <- matrix(nrow = nsim, ncol = 20)

mu_MC_vector <- numeric(100)
num_fail <- numeric(100)

total_time <- system.time({
  for (i in 1:100) {
    set.seed(i)

    for (j in 1:nsim) {
      X[j, ] <- rbinom(20, 1, 1 - p)
    }
    # number of failure
    num_fail[i] <- sum(apply(X, 1, is_connected))

    # mu_MC

```

```

    mu_MC_vector[i] <- num_fail[i] / nsim
  }
})

#mu_MC_vector
total_time;

## 사용자 시스템 elapsed
## 617.05    4.83 1594.30

mean(num_fail);

## [1] 1.3

mean(mu_MC_vector);

## [1] 1.3e-05

var(mu_MC_vector)

## [1] 1.727273e-10

## IS with 엄수연's h(x)

nsim <- 10^5
p_star <- 0.2
X_star <- matrix(nrow = nsim, ncol = 20)

mu_IS_star_vector <- numeric(100)
num_fail <- numeric(100)

total_time <- system.time({
  for (i in 1:100) {
    set.seed(i)

    for (j in 1:nsim) {
      X_star[j, ] <- rbinom(20, 1, 1 - p_star)
    }

    b_star <- apply(X_star, 1, function(x){ length(x[x==0])})
    w_star <- ((1-p)/(1-p_star))^20*((p*(1-p_star))/(p_star*(1-p)))^b_star
    h_star <- apply(X_star, 1, myroute)

    # number of failure
    num_fail[i] <- sum(h_star)

    # mu_IS_star
    mu_IS_star_vector[i] <- sum(h_star * w_star) / nsim
  }
})

#mu_IS_star_vector;
total_time;

```

```

## 사용자 시스템 elapsed
## 319.69    3.30  824.08

mean(num_fail);

## [1] 484.32

mean(mu_IS_star_vector);

## [1] 1.351035e-05

var(mu_IS_star_vector)

## [1] 3.845595e-12

## IS with my h(x) (is_connected(x))

nsim <- 10^5
p_star <- 0.2
X_star <- matrix(nrow = nsim, ncol = 20)

mu_IS_star_vector <- numeric(100)
num_fail <- numeric(100)

total_time <- system.time({
  for (i in 1:100) {
    set.seed(i)

    for (j in 1:nsim) {
      X_star[j, ] <- rbinom(20, 1, 1 - p_star)
    }

    b_star <- apply(X_star, 1, function(x){ length(x[x==0])})
    w_star <- ((1-p)/(1-p_star))^20*((p*(1-p_star))/(p_star*(1-p)))^b_star
    h_star <- apply(X_star, 1, is_connected)

    # number of failure
    num_fail[i] <- sum(h_star)

    # mu_IS_star
    mu_IS_star_vector[i] <- sum(h_star * w_star) / nsim
  }
})

## 사용자 시스템 elapsed
## 505.51    2.01 1445.75

mean(num_fail);

## [1] 484.32

```

```
mean(mu_IS_star_vector);  
## [1] 1.351035e-05  
var(mu_IS_star_vector)  
## [1] 3.845595e-12  
data <- data.frame(IS_star = mu_IS_star_vector, MC = mu_MC_vector)  
  
# boxplot  
boxplot(data, col = c("blue", "red"), names = c("IS", "MC"), main =  
"Comparison of IS and MC", ylab = "Value")
```